

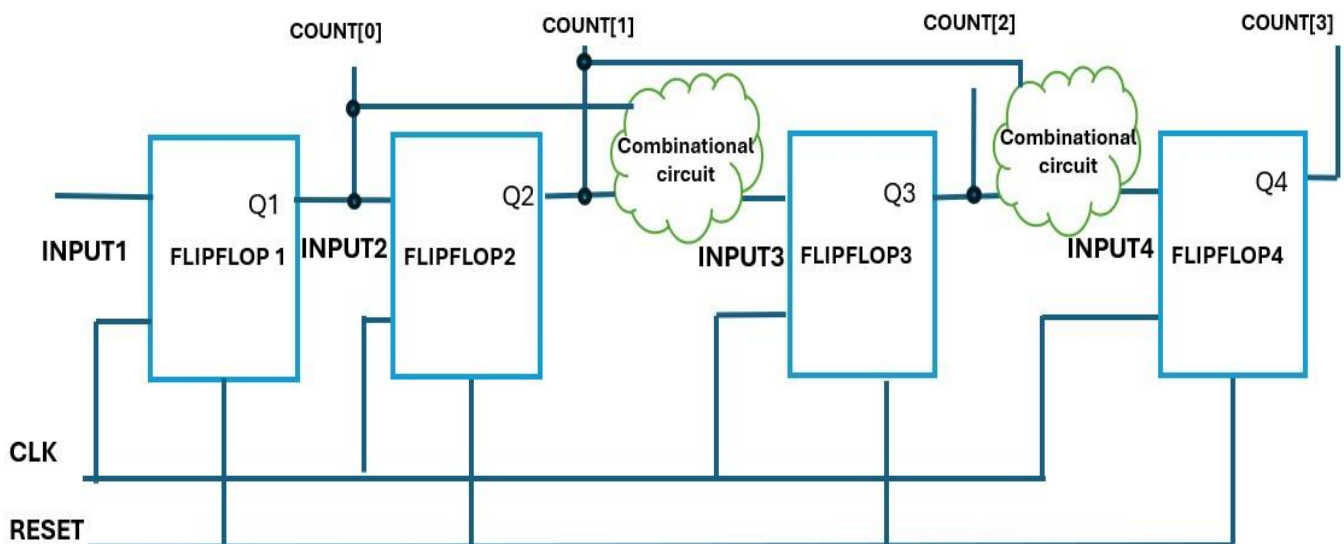
Simple 4-Bit UP Counter

Introduction:

A simple 4-bit up-counter is a fundamental digital circuit capable of counting and storing numerical values represented by four binary digits.

It is designed to increment (count up). This means it can count from Binary 4'b0000 (decimal 0) up to 4'b1111 (decimal 15), providing 16 distinct states in its counting cycle. Built using four interconnected flip-flops, the counter advances its stored binary count with each incoming clock pulse.

Block diagram



In the above block design, we can see a Verilog module named `up_counter_4bit` which implements a 4-bit simple up counter. This module has three ports: clock input(`clk`), input reset (asynchronous), and count, which is a 4-bit output register used to hold the current counter value. The always block is sensitive to the positive edge of both `clk` and `reset`, meaning the logic inside this block will execute whenever the clock signal rises or when the reset signal goes from low to high. Inside the block, a condition checks if the reset is active; if it is, the counter is immediately set to 0000, clearing the count. If the reset is not active, the counter increments by 1 on each rising edge of the clock. Since the counter is 4 bits wide, it can count from 0000 (decimal 0) to 1111 (decimal 15), and then wrap again to 0000, continuing in a loop.

Truth table

Clock Pulse	Q4 (MSB)	Q3	Q2	Q1 (LSB)	Binary	Decimal
0	0	0	0	0	0000	0
1	0	0	0	1	0001	1
2	0	0	1	0	0010	2
3	0	0	1	1	0011	3
4	0	1	0	0	0100	4
5	0	1	0	1	0101	5
6	0	1	1	0	0110	6
7	0	1	1	1	0111	7
8	1	0	0	0	1000	8
9	1	0	0	1	1001	9
10	1	0	1	0	1010	10
11	1	0	1	1	1011	11
12	1	1	0	0	1100	12
13	1	1	0	1	1101	13
14	1	1	1	0	1110	14
15	1	1	1	1	1111	15
16	0	0	0	0	0000	0 (Repeat)

Application:

These versatile components are crucial in various digital systems, finding applications in areas such as timekeeping for digital clocks, frequency division, event tracking, and creating sequences in timers and controllers.

Procedure to create project:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir simple_up_counter`
- 1.4: Open project directory: `cd simple_up_counter`

Step 2: Create Design file

- 2.1: Open `gvim` editor for design with command:
`gvim up_counter_4bit.v`

RTL Design Code:

```
module up_counter_4bit (input clk,
                        input reset,
                        output reg [3:0]count);
always @(posedge clk or posedge reset)
    begin
    if(reset)
    count <= 4'b0000;
    else
        count <= count + 1;
    end
endmodule
```

2.2: Save the file using command- :wq!

Step 3: Create Testbench file.

3.1: Open gvim editor for testbench with command :

gvim up_counter_4bit_tb.v

Testbench:

```
module up_counter_tb;
    reg clk;        reg reset;
    wire [3:0] count;

    up_counter_4bit uut (
        .clk(clk),
        .reset(reset),
        .count(count)
    );

    // clock generation
    always #5 clk = ~clk; // 10 time unit clock period
    initial
    begin
        $monitor("Time=%t|Reset=%b |count=%b", $time, reset, count);
        reset = 1; #10;
        reset = 0; #200;
        $finish;
    end
    initial begin
        $dumpfile ("up_counter_4bit.vcd");
        $dumpvars();
    end
endmodule
```

3.2: Save the testbench file using command- :wq!

3.3: Check the saved file in current directory : use command "ls"

Step 4: : Simulate the Design with Verilator

4.1: Use the following command to simulate

```
verilator --binary -j 0 -Wall up_counter_4bit.v
up_counter_4bit_tb.v --top up_counter_tb --timing --CFLAGS "-std=c++20" --trace
```

4.2: If any error is available, resolve it.

4.3: After simulation a new directory obj_dir will be created by tool, check it by using command "ls".

4.4: Open obj_dir

4.5: Use command: make -f Vup_counter_tb.mk Vup_counter_tb

4.6: A new directory with name Vup_counter_tb will be created

4.7: Use command ./Vup_counter_tb to compile

4.8: Check output at terminal

Simulation output:

```
Time=      0 | Reset=1 | count=0000
Time=     10 | Reset=0 | count=0000
Time=     15 | Reset=0 | count=0001
Time=     25 | Reset=0 | count=0010
Time=     35 | Reset=0 | count=0011
Time=     45 | Reset=0 | count=0100
Time=     55 | Reset=0 | count=0101
Time=     65 | Reset=0 | count=0110
Time=     75 | Reset=0 | count=0111
Time=     85 | Reset=0 | count=1000
Time=     95 | Reset=0 | count=1001
Time=    105 | Reset=0 | count=1010
Time=    115 | Reset=0 | count=1011
Time=    125 | Reset=0 | count=1100
Time=    135 | Reset=0 | count=1101
Time=    145 | Reset=0 | count=1110
Time=    155 | Reset=0 | count=1111
Time=    165 | Reset=0 | count=0000
Time=    175 | Reset=0 | count=0001
Time=    185 | Reset=0 | count=0010
Time=    195 | Reset=0 | count=0011
Time=    205 | Reset=0 | count=0100
```

The output shown is the simulation result of a 4-bit up counter implemented in Verilog. At the beginning of the simulation, the reset signal is high (Reset=1), which causes the counter to initialize to 0000. This confirms that the reset is asynchronous, as the counter is immediately set without waiting for a clock edge. Once the reset signal goes low (Reset=0), the counter begins incrementing its value on every rising edge of the clock. The count value increases from 0000 to 1111, representing decimal values from 0 to 15.

After reaching 1111, the counter wraps around and starts again from 0000, continuing the counting sequence. This behavior repeats, demonstrating that the design is functioning correctly as a 4-bit binary up counter with asynchronous reset.

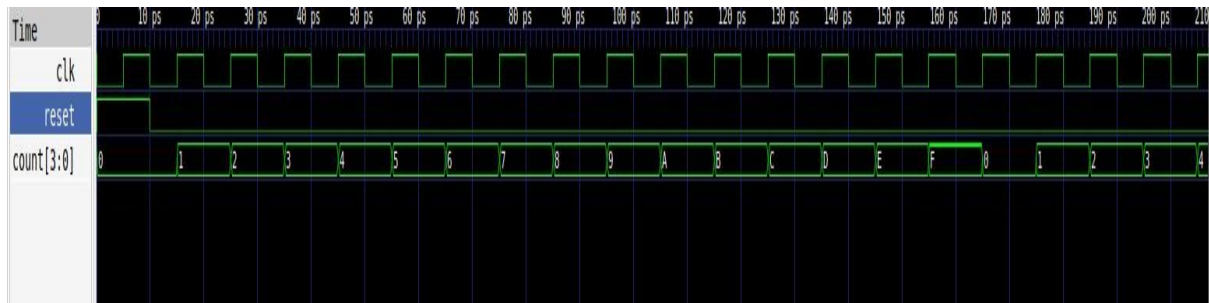
Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file:

```
gtkwave up_counter_4bit.vcd
```

5.2: Analyze the waveform to verify the correct operation of the 4-bit up counter

Output Waveform:



The output waveform shows the expected behavior of a 4-bit up counter with an asynchronous reset. At the beginning, the reset signal is high, keeping the counter output at 0000. As soon as the reset goes low, the counter starts incrementing its value on every rising edge of the clock. The count output increases sequentially from 0000 to 1111, and then wraps around to 0000, continuing the counting cycle. This confirms that the counter design is functioning correctly, with proper asynchronous reset and 4-bit binary counting.